

LINUX INTERGALÁCTICO: BASE DA
JORNADA JEDI

BASH, PRA QUE TE QUERO?



3

LISTA DE FIGURAS

Figura 1 – Terminal de comandos Microsoft Power Shell	6
Figura 2 – Opções de configuração do Microsoft Word	7
Figura 3 – O belíssimo gerenciador de janelas Gnome	8
Figura 4 – <i>Prompt</i> de comandos bash	10
Figura 5 – <i>Prompt</i> de comandos bash	12
Figura 6 – Exemplo do comando su sendo utilizado para tornar-se root	13
Figura 7 – Exemplo do comando “su” sendo utilizado para tornar-se outro usuário	13
Figura 8 – Exemplo do comando apt com o parâmetro search	15
Figura 9 – Exemplo do comando apt com o parâmetro install (1)	15
Figura 10 – Exemplo do comando apt com o parâmetro install (2)	16
Figura 11 – Exemplo de comando utilizando sudo	18
Figura 12 – Exemplo de comando passwd em execução	19
Figura 13 – Exemplo de comando useradd em execução	19
Figura 14 – Exemplo de comando groupadd em execução	20
Figura 15 – Exemplo de comando usermod em execução	21
Figura 16 – Exemplo de comando ls (simples)	23
Figura 17 – Exemplo de comando ls com parâmetros “l” e “a”	23
Figura 18 – Exemplo de comando ls com parâmetros “l”, “a” e “h” e diretório /	25
Figura 19 – Exemplo de comando pwd	25
Figura 20 – Exemplo de comando cd	26
Figura 21 – Exemplo de comando mkdir	27
Figura 22 – Exemplo de comando cp copiando diretório	31
Figura 23 – Exemplo de comando cat exibindo o conteúdo do arquivo /proc/locks	32
Figura 24 – Exemplo de comando head exibindo as dez primeiras linhas do arquivo /proc/cpuinfo	33
Figura 25 – Exemplo de comando tail exibindo as dez últimas linhas do arquivo /var/log/messages	33
Figura 26 – Exemplo de comando tail acompanhando /var/log/syslog com o parâmetro -f	34
Figura 27 – Exemplo de comando less exibindo o arquivo /var/log/messages	34
Figura 28 – Exemplo de comando locate usado para localizar o arquivo Imagem1.png	35
Figura 29 – Exemplo de comando grep buscando a palavra fiap dentro do arquivo /var/log/auth.log	36
Figura 30 – Exemplo de comando grep buscando o termo FIAP ON em /home	36
Figura 31 – Exemplo de comando tar empacotando e comprimindo arquivos	38
Figura 32 – Exemplo de comando tar empacotando e comprimindo arquivos	39
Figura 33 – Exemplo de comando wget baixando arquivos da Internet	40
Figura 34 – Exemplo de comando wget retomando um download pela metade	41

LISTA DE QUADROS

Quadro 1 – Diretórios em um sistema Linux	22
---	----

EMSE

LISTA DE PROMPTS DE COMANDO

Comando de prompt.1 – Exemplo do comando man	11
Comando de prompt 2 – Exemplo de uso do parâmetro – Help.....	12
Comando de prompt 3 – Exemplo de apt sendo utilizado para atualizar repositórios (1).....	14
Comando de prompt 4 – Exemplo de apt sendo utilizado para atualizar repositórios (2).....	14
Comando de prompt 5 – Exemplo de apt sendo utilizado para instalar o pacote sudo	16
Comando de prompt 6 – Adicionando o usuário hpoyatos ao grupo sudo	17
Comando de prompt 7 – Exemplo de uso do parâmetro -p no comando mkdir	27
Comando de prompt 8 – Exemplo do comando rm apagando um arquivo	28
Comando de prompt 9 – Exemplo do comando rm apagando um diretório vazio	28
Comando de prompt 10 – Exemplo do comando rm apagando um diretório cheio...	29
Comando de prompt 11 – Exemplo do comando cp copiando um arquivo	30
Comando de prompt 12 – Exemplo do comando cp copiando um arquivo, renomeando-o	30
Comando de prompt 13 – Exemplo do comando mv usado para mover um diretório	31
Comando de prompt 14 – Exemplo do comando mv usado para renomear um arquivo	32
Comando de prompt 15 – Exemplo do comando unzip descompactando arquivos zip	37
Comando de prompt 16 – Exemplo do comando unzip testando um arquivo zip.....	37
Comando de prompt 17 – Exemplo do comando tar empacotando e comprimindo arquivos txt	39

SUMÁRIO

BASH, PRA QUE TE QUERO?	6
1 INTRODUÇÃO	6
2 POR QUE APRENDER COMANDOS EM UM MUNDO DE INTERFACES GRÁFICAS?	6
3 ALGUMAS CONSIDERAÇÕES ANTES DE COMEÇAR	9
4 COMANDOS ESSENCIAIS	11
4.1 man ou parâmetro – Help	11
4.2 su – Tornando-se outro usuário	12
4.3 apt – Gerenciamento de pacotes de um sistema .deb	14
4.4 sudo – Rodando comandos como se fosse o superusuário	17
5 GERENCIAMENTO DE USUÁRIOS	18
5.1 passwd – Modificar a senha do usuário	19
5.2 useradd – Adicionar um usuário	19
5.3 groupadd – Adicionar um grupo	20
5.4 usermod – Modificando um usuário	20
6 MANIPULAÇÃO DE ARQUIVOS E PASTAS	21
6.1 Raiz de diretórios	21
6.2 ls – Listar arquivos e diretórios	23
6.3 pwd – Mostrar diretório ou pasta atual	25
6.4 cd – Mudar de diretório ou pasta	26
6.5 mkdir – Criar um diretório	27
6.6 rm – Apagar um arquivo ou diretório	28
6.7 cp – Copiando arquivos e diretórios	30
6.8 mv – Movendo ou renomeando arquivos e diretórios	31
6.9 cat – Exibe o conteúdo de um arquivo	32
6.10 head – Exibir linhas iniciais de um arquivo	32
6.11 tail – Exibir linhas finais de um arquivo	33
6.12 less – Exibe o conteúdo de um arquivo	34
6.13 locate – Localiza arquivos	35
6.14 grep – Localiza termos dentro de arquivos	35
6.15 unzip – Descompactar arquivos zip	37
6.16 tar – (Des)empacotar e (des)comprimir arquivos	37
6.17 wget – Baixando arquivos da Internet	40
CONCLUSÃO	41
REFERÊNCIAS	42
GLOSSÁRIO	43

BASH, PRA QUE TE QUERO?

1 INTRODUÇÃO

Aprender comandos para o sistema operacional Linux é indispensável a qualquer profissional de TI, seja este um programador, administrador de rede ou um especialista em segurança. Neste capítulo, abordaremos os principais comandos deste sistema operacional, formando um verdadeiro “canivete suíço” para desbravar o sistema do pinguim.

Sem o conhecimento dos comandos abordados a seguir, o profissional fica extremamente limitado em relação às possibilidades de configuração deste sistema, além do conhecimento de como proteger seu ambiente computacional.

2 POR QUE APRENDER COMANDOS EM UM MUNDO DE INTERFACES GRÁFICAS?

O título desta seção é provocativo, e talvez seja uma dúvida genuína: Por que devo aprender comandos Linux em um mundo dominado por interfaces gráficas? Este tipo de questionamento é muito comum, especialmente vindo de usuários do sistema operacional Windows.

Em primeiro lugar, o terminal de comandos anteriormente conhecido como MS-DOS nunca foi abandonado pelo sistema Windows; na verdade, este sistema conta com uma nova versão deste terminal de comandos, o **Power Shell**.

```
PS C:\> Get-AzureRmADServicePrincipal -SearchString "AzureKeysRollerDaemon"

DisplayName      Type      ObjectID
-----
AzureKeysRollerDaemon 672f1afa-526a-4ef6-819c-975c7cd79022

PS C:\> New-AzureRmRoleAssignment -ObjectID 672f1afa-526a-4ef6-819c-975c7cd79022 -RoleDefinitionName Contributor -Scope /subscriptions/c276fc76-9cd4-44c9-99a7-4fd71546436e

RoleAssignmentId : /subscriptions/c276fc76-9cd4-44c9-99a7-4fd71546436e/providers/Microsoft.Author
                  ization/roleAssignments/9182ae32-4219-4c26-a80b-891ba649ae36
Scope             : /subscriptions/c276fc76-9cd4-44c9-99a7-4fd71546436e
DisplayName       : AzureKeysRollerDaemon
SignInName        :
RoleDefinitionName : Contributor
RoleDefinitionId   : b24988ac-6180-42a0-ab88-20f7382dd24c
ObjectID          : 672f1afa-526a-4ef6-819c-975c7cd79022
ObjectType         : ServicePrincipal

PS C:\>
```

Figura 1 – Terminal de comandos Microsoft Power Shell
Fonte: Microsoft Docs WebSite (2018)

Existem situações nas quais o uso de um terminal de comandos se torna mais prático do que a interface gráfica. Quer um exemplo? Você já teve curiosidade de acessar as opções de configuração de seu Microsoft Word? (Disponível no menu Arquivo, item “Opções”). Ele possui várias seções de configurações, e cada uma delas conta com uma infinidade de caixas de textos, caixas de seleção e botões.

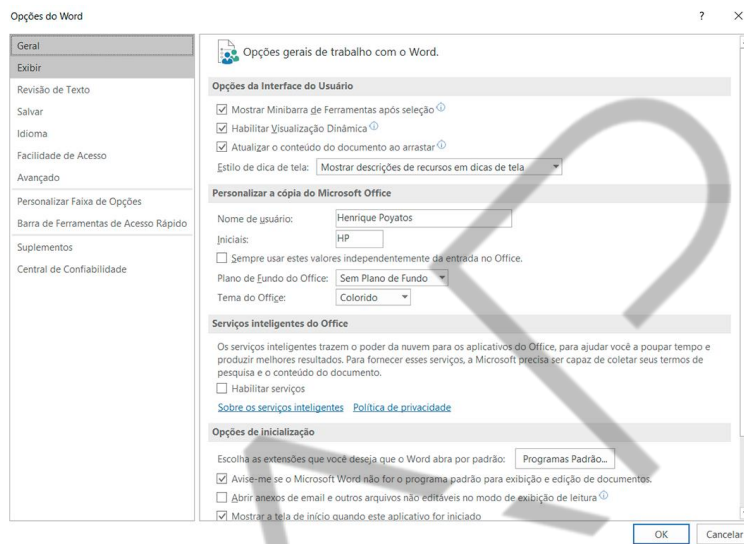


Figura 2 – Opções de configuração do Microsoft Word
Fonte: Elaborado pelo autor (2018)

Em um terminal de comandos, estas várias possibilidades de configuração do Microsoft Word seriam substituídas por um único comando com várias opções de parametrização. Assim, desde que você saiba o parâmetro correto, poderia alterar a configuração rapidamente, enquanto na interface gráfica várias telas teriam que ser roladas até se localizar a configuração desejada. Talvez você indague, com uma certa razão, que decorar os parâmetros de um comando não é muito fácil. Sim, é verdade, mas, no decorrer deste capítulo, você verá que isso não é necessário, porque aprenderá como procurar ajuda em caso de necessidade.

Também é fato que o sistema operacional Linux conta com belíssimos e eficazes softwares de gerenciamento de janelas (*Windows Managers*, vulgarmente conhecidos como interface gráfica), como é o caso do KDE, Gnome, XFCE, Window Maker e vários outros. No entanto, também é realidade que a maioria das implementações de servidor Linux **NÃO CONTA COM UMA INTERFACE GRÁFICA.**

Na verdade, para muitos, a implementação de um servidor X e um gerenciador de janelas representa um imenso desperdício dos recursos do servidor (lê-se aqui processador e memória, recursos valiosos em um servidor).

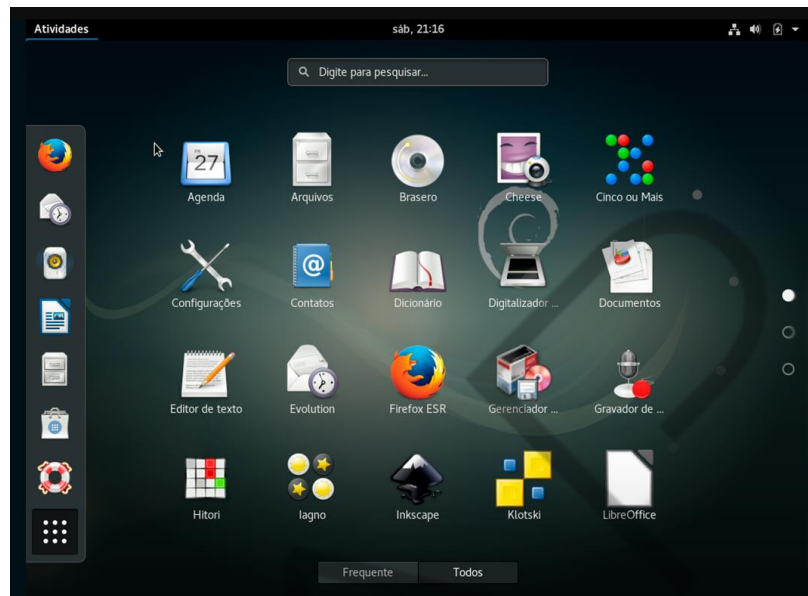


Figura 3 – O belíssimo gerenciador de janelas Gnome
Fonte: Elaborado pelo autor (2018)

O terminal de comandos será sempre a forma de interação com um *Kernel* de sistema operacional mais rápida e leve que existe: não é necessária uma placa gráfica poderosa, grandes quantidades de memória RAM ou mesmo um monitor colorido. Na verdade, o terminal dispensa o uso de um monitor, possibilitando um acesso remoto por meio de SSH, cujo tráfego de dados será sempre infinitamente menor do que um TeamViewer (software que permite acessar interfaces gráficas remotamente), ou seja, nem sequer um bom link de dados é necessário para acessar o terminal de comandos de um servidor remoto.

Sendo assim, desarme-se de qualquer tipo de preconceito a partir de agora: deixe suas armas por aqui e siga conosco adiante nesta jornada, onde abordaremos os comandos mais essenciais presentes no sistema operacional Linux. Talvez seu receio seja o desconhecido, mas nada que uma certa dose de dedicação e prática destes comandos não vença. Vamos lá!

3 ALGUMAS CONSIDERAÇÕES ANTES DE COMEÇAR

A variedade de opções disponíveis no software livre é seu grande trunfo, mas, ao mesmo tempo, um calcanhar de Aquiles: se, por um lado, a liberdade oriunda do software de código aberto permite uma infinidade de projetos diferentes, esta mesma variedade pode ser intimidadora para um iniciante.

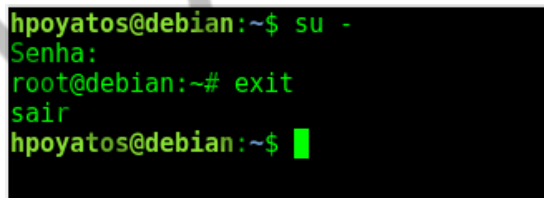
Em primeiro lugar, como vimos no capítulo anterior (capítulo 10, intitulado “A marcha dos pinguins”) o Linux possui diversas versões ou sabores diferentes: Red Hat, Fedora, Debian, Ubuntu, Slackware, Gentoo, Arch Linux, existem dezenas de boas opções. A verdade é que, depois que você começa a conhecer um pouco cada uma delas, descobrirá que elas não são tão diferentes assim. Isso graças a uma iniciativa na década de 2000, conhecida como **Linux Standard Base (LSB)**, que estabeleceu uma especificação que manteve similaridades entre as distribuições, evitando o que havia acontecido com o Sistema Operacional Unix no passado, cujas distribuições se tornaram muito, muito diferentes. Aliás, algumas delas são até mesmo “aparentadas”, como é o caso do Debian e o Ubuntu, os quais poderíamos chamar aqui de distribuição “mãe” e “filha”.

Exatamente por ser a “distribuição mãe” de dezenas de opções disponíveis, e por se manter como uma distribuição absolutamente gratuita e mais livre possível (o que ajuda em sua adoção no mercado), vamos optar pela distribuição Debian Linux para este e outros capítulos (como o capítulo 14, desta fase; e o capítulo 5, da Fase 2 deste curso).

Acredite se quiser, mas existem também dezenas de terminais de comandos diferentes no Linux: bash, csh, zsh, entre vários outros. O terminal de comandos **bash** é uma implementação livre do tradicional terminal de comandos do Unix, o *shell*, ou simplesmente “sh”. O nome do terminal, bash, é um acrônimo para **Bourne-Again Shell** ou, em uma tradução livre, “*shell* renascido” (ou “concha renascida”, mas aí fica estranho). A verdadeira fênix dos terminais de comandos!

Além de ser o clássico terminal do Unix renascido (diria que o adjetivo certo deveria ser “evoluído”), existe uma boa razão para adotá-lo: 99% de todos os usuários o utilizam; todas as distribuições o instalam por padrão e mesmo aqueles que gostam de outros terminais sempre mantêm o **bash** instalado como opção, ou seja, é praticamente certo que ele sempre estará disponível para uso. Sendo assim, a maioria dos comandos que você aprenderá neste capítulo funciona em outros sabores de Linux e também nos Unix. Nada mal, não é?

O *prompt* de comandos é simbolizado por alguns caracteres iniciais que indicam que o terminal está pronto e aguardando pelo comando do usuário. No MS-DOS, era a letra do drive e o diretório em que o usuário está, exemplo: “C:\Windows>”. No bash, geralmente este *prompt* é o nome do usuário, arroba (“@”), o nome da máquina (ou hostname), dois-pontos (“:”), o diretório em que o usuário está, terminado pelo símbolo de cifrão (“\$”). Quando o usuário em questão for o administrador do sistema (superusuário ou *root*), o símbolo de cifrão se transforma em uma cerquinha (“#”, sim, cada um chama isso de um jeito, jogo da velha, sustenido, *hash*, enfim):



```
hpoyatos@debian:~$ su -
Senha:
root@debian:~# exit
sair
hpoyatos@debian:~$
```

Figura 4 – *Prompt* de comandos bash
Fonte: Elaborado pelo autor (2018)

Para simplificar, vamos encurtá-lo no material didático: o *prompt* será apenas “\$” para o usuário comum e o “#” quando o usuário for o *root*, exceto nas imagens que representarão execuções legítimas.

Outra observação importante: um terminal de comandos Linux é sempre *case-sensitive*, ou seja, letras maiúsculas são diferentes de letras minúsculas. Sendo assim, “ls” é um comando que lista arquivos e diretório no Linux, enquanto “LS” simplesmente não existe. Muita atenção!

Por serem muitos, dividimos este capítulo em seções: comandos essenciais, manipulação de arquivos e pastas, gerenciamento de processos, configuração de rede, entre outros, assim, você poderá localizar rapidamente o que precisa.

4 COMANDOS ESSENCIAIS

Nesta seção, relacionaremos os comandos básicos essenciais para se começar. Sem excessos ou frescura, vamos começar!

4.1 man ou parâmetro – Help

O primeiro comando é o mais essencial de todos e uma lição para todos nós: **NÃO SABE? PEÇA AJUDA!** O comando **man** é a abreviação de **manual**, ou seja, quando digitamos o comando **man** seguido do comando que desejamos aprender, ele nos mostra o manual completo dele: para que serve, quais são os parâmetros e, em alguns casos, até exemplos de uso! Se quisermos, por exemplo, saber como funciona o clássico comando “ls”, basta digitar:

```
$ man ls
```

Comando de prompt.1 – Exemplo do comando man
Fonte: Elaborado pelo autor (2018)

E temos como resultado o manual do comando ls, com uma breve descrição do que faz o comando, sinopse, descrição detalhada, parâmetros e exemplos de uso. Se o Linux foi instalado em português (ou se os pacotes de idiomas pt-br foram instalados posteriormente), o manual vem em nosso idioma!

```

LS(1)                                General Commands Manual                                LS(1)

NOME
    ls, dir, vdir - lista o conteúdo do diretório

SINOPSE
    ls [opções] [arquivo...]
    dir [arquivo...]
    vdir [arquivo...]

    Opções POSIX: [-CFRacdilqrutl]

    Opções GNU (forma reduzida): [-labcdfghiklmnopqrstuvwABCDFGHLNORSUX] [-w
coluna] [-T coluna] [-I modelo] [--full-time] [--show-control-chars]
[--block-size=size]
[--format={long,verbose,commas,across,vertical,single-column}]
[--sort={none,time,size,extension}]
[--time={atime,access,use,ctime,status}] [--color={none,auto,always}]
[--help] [--version] [--]

DESCRIÇÃO
    O programa ls lista primeiramente seus argumentos que não sejam arquivos de
diretórios, e, então, para um argumento diretório, todos os arquivos
listáveis incluídos dentro daquele diretório. Se nenhum argumento de opção
Manual page ls(1) line 1 (press h for help or q to quit)

```

Figura 5 – *Prompt* de comandos bash
 Fonte: Elaborado pelo autor (2018)

O comando de manual “prende o terminal”, portanto, navegue pelo manual com as setas para cima e para baixo pelo tempo que quiser e, quando estiver satisfeito, pressione a letra “q” (*quit*) para sair, e o comando será encerrado. Outra possibilidade é digitar o comando desejado seguido por `--help` (traço-traço help):

```
$ ls --help
```

Comando de prompt 2 – Exemplo de uso do parâmetro – Help
 Fonte: Elaborado pelo autor (2018)

Uma versão resumida do manual é apresentada e o *prompt*, liberado. Utilize as teclas [CTRL]+[PAGE UP] para subir e ler a saída, enquanto [CTRL]+[PAGE DOWN] rola o texto para baixo.

4.2 su – Tornando-se outro usuário

Por razões de segurança, é recomendável que o usuário sempre opere com seu próprio usuário, com permissões restritas. Já falamos sobre isso antes: por excesso de cansaço, sono ou mesmo confiança, podemos cometer erros. E isso acontece até com os mais preparados: em 1968, o astronauta Jim Lovell apertou um botão que não deveria ser apertado durante a missão da Apollo 8, embora a NASA tenha afirmado que “astronautas são treinados para nunca cometerem erros”.

Coube a Margaret Hamilton (cuja história foi brevemente contada no segundo capítulo desta fase, intitulado “E o mercado de trabalho, é bom?”) e seu time salvar a missão e a vida dos astronautas. A partir daquele momento, Hamilton começou a pensar sistemas como “parte *software*, parte *hardware*, parte GENTE” (GEORGE, 2019).

Voltando à nossa pauta, espero ter convencido você a manter suas próprias permissões restritas. No entanto, em alguns momentos, é necessário se tornar outro usuário, costumeiramente, o superusuário root, aquele-que-pode-tudo.

O comando `su` é o empregado para a mudança de usuário. Em geral, ele é atribuído como sendo o comando “para se tornar root”, mas, na realidade, ele permite que nos tornemos qualquer usuário do sistema, basta digitar `su` e, na sequência, o nome do usuário que queremos ser. Caso nada seja digitado, o padrão é, sim, vira super.

```
hpoyatos@debian:~$ su
Senha:
root@debian:/home/hpoyatos#
```

Figura 6 – Exemplo do comando `su` sendo utilizado para tornar-se root
Fonte: Elaborado pelo autor (2018)

Por padrão, o comando `su` solicitará a senha do usuário que você deseja se tornar. Se informada corretamente, o *prompt* muda de usuário antes do símbolo @ (arroba). Se o *prompt* estiver em sua versão simples e não deixar o usuário explícito, você pode usar ainda o comando `whoami` (*Who am I?* ou “Quem sou eu?” em bom português, embora sempre que eu rode este comando me sinta meio senil).

```
hpoyatos@debian:~$ su
Senha:
root@debian:/home/hpoyatos# whoami
root
root@debian:/home/hpoyatos# su outrousuário
outrousuário@debian:/home/hpoyatos$ whoami
outrousuário
outrousuário@debian:/home/hpoyatos$
```

Figura 7 – Exemplo do comando “`su`” sendo utilizado para tornar-se outro usuário
Fonte: Elaborado pelo autor (2018)

No exemplo da Figura “Exemplo do comando su sendo utilizado para tornar-se outro usuário”, o comando “su” foi usado para tornar-se um usuário chamado “outrousuário”. Novamente, com o prompt utilizado no exemplo, o comando embora pareça meio inútil (ok, totalmente inútil), em algumas situações, pode estar em sua versão simplificada, simplesmente “\$” ou “#”. Repare também que fornecer a senha deste usuário não é necessário, pois, com “o todo-poderoso” **root**, tudo é possível.

4.3 apt – Gerenciamento de pacotes de um sistema .deb

Já que nos tornamos o root, vamos aproveitar a oportunidade para mostrar como operar o comando apt, responsável pelo gerenciamento de pacotes em um sistema do tipo .deb (Debian e seus descendentes, tantos quanto do Mr. Catra). Este comando não funciona em sistemas baseados em .rpm (Red Hat, Fedora, CenOS, OpenSUSE etc.) e outros (como Slackware ou Gentoo).

```
# apt update
```

Comando de prompt 3 – Exemplo de apt sendo utilizado para atualizar repositórios (1)
Fonte: Elaborado pelo autor (2018)

O comando apresentado na Comando de prompt # **apt update** atualiza a referência dos repositores, ou seja, ele baixa a lista de pacotes disponíveis para download com suas versões. Com isso, o sistema pode comparar as versões instaladas com as disponíveis, permitindo determinar quais pacotes poderiam ser atualizados.

```
# apt upgrade
```

Comando de prompt 4 – Exemplo de apt sendo utilizado para atualizar repositórios (2)
Fonte: Elaborado pelo autor (2018)

O comando na Comando de prompt # **apt upgrade** exibe a lista de pacotes que estão atualizados e confirma se desejamos realmente atualizá-los. Em caso positivo, apt baixa os pacotes e atualiza-os automaticamente.

O comando apt possui um parâmetro “search”, que permite buscar por pacotes antes de instalá-los. Observe o que acontece quando rodo o comando procurando pela palavra “trezor” (Curiosidade: trezor é uma carteira do tipo hardware para criptomoedas):

```
root@debian:/home/hpoyatos# apt search trezor
Sorting... Pronto
Full Text Search... Pronto
python-keepkey/stable 0.7.3-1 all
  library for communicating with KeepKey Hardware Wallet
python-trezor/stable 0.7.6-1 all
  library for communicating with TREZOR Bitcoin HW wallet
root@debian:/home/hpoyatos#
```

Figura 8 – Exemplo do comando apt com o parâmetro search
Fonte: Elaborado pelo autor (2018)

Já o parâmetro “install” propicia instalar pacotes disponíveis nos repositórios. Para que a operação seja feita com sucesso, você precisa estar logado como o superusuário root.

```
root@debian:/home/hpoyatos# apt install sudo
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os seguintes pacotes foram instalados automaticamente e já não são necessários:
  libhidapi-hidraw0 libhidapi-libusb0 python-ecdsa python-hid python-mnemonic
  python-pbkdf2 python-protobuf
Utilize 'apt autoremove' para os remover.
Os NOVOS pacotes a seguir serão instalados:
  sudo
0 pacotes atualizados, 1 pacotes novos instalados, 0 a serem removidos e 0 não a
tualizados.
É preciso baixar 1.055 kB de arquivos.
Depois desta operação, 3.108 kB adicionais de espaço em disco serão usados.
Obter:1 http://ftp.br.debian.org/debian stretch/main amd64 sudo amd64 1.8.19p1-2
.1 [1.055 kB]
Baixados 1.055 kB em 2s (425 kB/s)
A seleccionar pacote anteriormente não seleccionado sudo.
(Lendo banco de dados ... 129928 ficheiros e directórios actualmente instalados.
)
A preparar para descompactar .../sudo_1.8.19p1-2.1_amd64.deb ...
A descompactar sudo (1.8.19p1-2.1) ...
Configurando sudo (1.8.19p1-2.1) ...
A processar 'triggers' para systemd (232-25+deb9u1) ...
A processar 'triggers' para man-db (2.7.6.1-2) ...
root@debian:/home/hpoyatos#
```

Figura 9 – Exemplo do comando apt com o parâmetro install (1)
Fonte: Elaborado pelo autor (2018)

A Figura “Exemplo do comando apt com o parâmetro install” mostra a instalação do pacote **sudo**. Rode este comando também, pois **sudo** não está disponível por padrão no sistema Debian, e trata-se de um comando muito útil.

```
# apt install sudo
```

Comando de prompt 5 – Exemplo de apt sendo utilizado para instalar o pacote sudo

Fonte: Elaborado pelo autor (2018)

Por fim, o parâmetro “remove” pode ser utilizado para desinstalar pacotes. Observe o que acontece quando executo o comando para desinstalar `python-trezor` (claro que o pacote precisa ser instalado primeiro com `apt install python-trezor`):

```
root@debian:/home/hpoyatos# apt remove python-trezor
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os seguintes pacotes foram instalados automaticamente e já não são necessários:
 libhidapi-hidraw0 libhidapi-libusb0 python-ecdsa python-hid python-mnemonic
 python-pbkdf2 python-protobuf
Utilize 'apt autoremove' para os remover.
Os pacotes a seguir serão REMOVIDOS:
 python-trezor
0 pacotes atualizados, 0 pacotes novos instalados, 1 a serem removidos e 0 não a
tualizados.
Depois desta operação, 329 kB de espaço em disco serão liberados.
Você quer continuar? [S/n] s
(Lendo banco de dados ... 129959 ficheiros e directórios actualmente instalados.
)
A remover python-trezor (0.7.6-1) ...
root@debian:/home/hpoyatos#
```

Figura 10 – Exemplo do comando apt com o parâmetro install (2)

Fonte: Elaborado pelo autor (2018)

Como pode ser observado nos exemplos, os repositórios das distribuições Linux contam com uma completa rede de dependências: quando um determinado pacote vai ser instalado e precisa de outros aplicativos, bibliotecas etc, o próprio comando rastreia as dependências e as instala automaticamente (e as remove quando não se tornam mais necessárias).

4.4 sudo – Rodando comandos como se fosse o superusuário

Tornar-se o usuário root e voltar a ser seu próprio usuário o tempo todo pode ser cansativo e, por descuido, o usuário pode ficar como **root** tempo demais. Assim sendo, o comando **sudo** pode ser uma das alternativas mais viáveis nestes casos. A primeira coisa é instalar o sudo com o comando **# apt install sudo** como o usuário root. Logo depois, rode o seguinte comando (também como root):

```
# usermod -aG sudo hpoyatos
```

Comando de prompt 6 – Adicionando o usuário hpoyatos ao grupo sudo

Fonte: Elaborado pelo autor (2018)

A Comando de prompt **# usermod -aG sudo hpoyatos** adiciona o usuário “hpoyatos” ao grupo de usuários sudo, que foi criado quando instalei o pacote. Troque “hpoyatos” pelo nome de usuário desejado, é claro. Por razões de segurança, apenas os membros do grupo **sudo** poderão usar o comando. O nome do grupo muda de distribuição Linux para distribuição. Em alguns casos, este grupo é conhecido como **wheel** (é o caso do meu amado Gentoo!). Conferir o arquivo `/etc/sudoers` com o comando **# cat /etc/sudoers** pode esclarecer este ponto.

Antes de rodar o comando seguinte, deslogue e logue novamente, pois a alocação de grupos do usuário é feita no início da sessão do usuário, seja isso em modo texto ou com a interface gráfica.

A sintaxe é **\$ sudo [comando que desejo rodar com privilégios]**; o terminal solicitará a senha do seu próprio usuário e, caso ele seja membro do grupo sudo, o comando será executado com os privilégios de superusuário:

```

hpoyatos@debian:~$ sudo apt install nmap
[sudo] senha para hpoyatos:
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os seguintes pacotes foram instalados automaticamente e já não são necessários:
  libhidapi-hidraw0 libhidapi-libusb0 python-ecdsa python-hid python-mnemonic
  python-pbkdf2 python-protobuf
Utilize 'sudo apt autoremove' para os remover.
The following additional packages will be installed:
  liblinear3 ndiff
Pacotes sugeridos:
  liblinear-tools liblinear-dev
Os NOVOS pacotes a seguir serão instalados:
  liblinear3 ndiff nmap
0 pacotes atualizados, 3 pacotes novos instalados, 0 a serem removidos e 0 não a
tualizados.
É preciso baixar 5.502 kB de arquivos.
Depois desta operação, 23,7 MB adicionais de espaço em disco serão usados.
Você quer continuar? [S/n] s
Obter:1 http://ftp.br.debian.org/debian stretch/main amd64 liblinear3 amd64 2.1
0+dfsg-2 [39,2 kB]
Obter:2 http://ftp.br.debian.org/debian stretch/main amd64 ndiff all 7.40-1 [286
kB]
Obter:3 http://ftp.br.debian.org/debian stretch/main amd64 nmap amd64 7.40-1 [5

```

Figura 11 – Exemplo de comando utilizando sudo
 Fonte: Elaborado pelo autor (2018)

Na Figura “Exemplo de comando utilizando sudo”, solicitei a instalação do pacote nmap, muito útil para verificar as portas de serviços abertas de um servidor. Sudo solicitou a senha do meu usuário (hpoyatos) e prosseguiu com a instalação como se meu usuário fosse o **root**.

O procedimento é mais seguro (e menos trabalhoso) do que tornar-se **root** o tempo todo com o comando **su**, neste caso, os privilégios são usados pontualmente, conforme a necessidade.

5 GERENCIAMENTO DE USUÁRIOS

Os comandos a seguir permitem realizar algumas operações, como trocar a senha do usuário, criar usuários ou grupos, assim como modificar usuários existentes.

5.1 passwd – Modificar a senha do usuário

Para modificar a senha do usuário, o comando **passwd** (abreviação da palavra inglesa *password* ou senha) pode ser utilizado. Ao digitar passwd no terminal, ele, por sua vez, solicita a senha atual e, posteriormente, a nova senha e a confirmação da nova senha. Repare que o terminal de comandos do Linux não exibe as senhas digitadas ou mesmo máscaras, como ****, ele não exibe absolutamente nada e um usuário de primeira viagem pode achar que existe algo errado (Relaxa, é assim mesmo!).

```
hpoyatos@debian:~$ passwd
Mudando senha para hpoyatos.
Senha UNIX (atual):
Digite a nova senha UNIX:
Redigite a nova senha UNIX:
passwd: senha atualizada com sucesso
hpoyatos@debian:~$
```

Figura 12 – Exemplo de comando passwd em execução
Fonte: Elaborado pelo autor (2018)

É possível trocar a senha de outros usuários, bastando colocar seu nome logo após o comando passwd e, neste caso, ele solicitará a senha deste primeiro, a não ser que você seja o root, aí, a senha atual não será solicitada. Sendo assim, **\$ sudo passwd root** pode ser usado para trocar a senha do próprio superusuário, portanto, muito cuidado a quem você concede altos privilégios.

5.2 useradd – Adicionar um usuário

Quando for necessário criar um usuário no sistema Linux, o comando **useradd** deve ser usado. Utilize os parâmetros “-d” para informar o diretório pessoal deste usuário, e “-p” para informar a senha (embora você possa rodar o comando **\$ sudo passwd [usuario]** logo na sequência):

```
hpoyatos@debian:~$ sudo useradd fiap -d /home/fiap -p senha_usuario_fiap
hpoyatos@debian:~$
```

Figura 13 – Exemplo de comando useradd em execução
Fonte: Elaborado pelo autor (2018)

Embora não fique explícito, por padrão, o comando **useradd** cria um **grupo com o mesmo nome do usuário e o coloca como único membro**. Procure outras informações sobre o comando com **\$ man useradd**, pois existem parâmetros para criar usuários temporários, adicioná-los previamente em outros grupos, entre outras possibilidades.

Aliás, criar usuários que expirem sozinhos é uma excelente prática para usuários temporários, pois são frequentes os casos de empresas que não os desativam, criando uma baita brecha de segurança.

5.3 groupadd – Adicionar um grupo

Agrupamento é uma prática interessante, pois podemos conceder acesso (ou revogar) a vários usuários de uma única vez. O comando **groupadd** permite criar um grupo de usuários e, para tal, basta digitar o comando **groupadd** acompanhado do nome deste novo grupo (e **sudo**, pois criar grupos não é para qualquer um):

```
hpyatos@debian:~$ sudo groupadd fiapon
hpyatos@debian:~$
```

Figura 14 – Exemplo de comando **groupadd** em execução
Fonte: Elaborado pelo autor (2018)

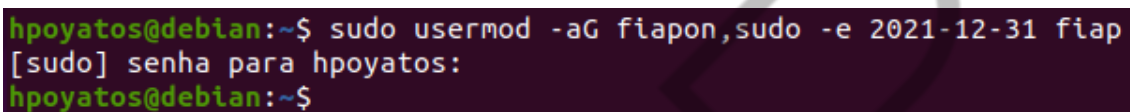
5.4 usermod – Modificando um usuário

Embora seja possível alterar os arquivos **/etc/passwd** (configurações do usuário), **/etc/shadow** (as senhas dos usuários) e **/etc/group** (configurações de grupo) diretamente com os privilégios de um **root**, é **uma péssima ideia**, pois qualquer falha na alteração destes arquivos, uma única vírgula fora do lugar, pode comprometer o acesso dos usuários ao sistema.

Sendo assim, o comando **usermod** (de *user modification* ou modificação de usuário) e os parâmetros “-aG” são geralmente usados em conjunto e permitem adicionar o usuário em novos grupos sem, no entanto, retirá-lo dos grupos dos quais já faz parte.

O parâmetro “-d” permite estabelecer um novo diretório pessoal para o usuário e “-L” (de *lock*) bloqueia um usuário, enquanto “-U” (de *unlock*) pode desbloqueá-lo facilmente. O uso de “-e” permite estabelecer uma data de expiração para a conta do usuário, que é automaticamente bloqueada após esta data (o que, conforme já disse, é uma excelente ideia do ponto de vista da segurança).

A Figura “Exemplo de comando usermod em execução” adiciona o recém-criado usuário **fiap** em dois novos grupos (**sudo** e **fiapon**) e estabelece que o usuário ficará ativo até o final do ano de 2021:



```
hpoyatos@debian:~$ sudo usermod -aG fiapon,sudo -e 2021-12-31 fiap
[sudo] senha para hpoyatos:
hpoyatos@debian:~$
```

Figura 15 – Exemplo de comando usermod em execução
Fonte: Elaborado pelo autor (2019)

6 MANIPULAÇÃO DE ARQUIVOS E PASTAS

Navegar pelo sistema de arquivos de um sistema operacional organizando arquivos em diretórios (ou pastas) é uma das necessidades primárias de um sistema operacional. Sendo assim, vejamos alguns comandos úteis nesta tarefa.

6.1 Raiz de diretórios

Antes de abordar os comandos, no entanto, precisamos falar um pouco sobre a raiz de diretórios do sistema operacional Linux. Como a maioria dos usuários conhece a árvore de um sistema Windows, tentarei explicar fazendo um paralelo entre eles, quando isso for possível, é claro.

Diretório	Descrição
/	É o diretório-raiz, onde tudo começa. Ele seria equivalente ao C:\ do Windows, mas diferente do sistema operacional de Redmond, um sistema *nix tem sempre uma raiz única; o Windows cria uma raiz (ou diretório inicial) para cada volume, sempre identificando com uma letra (D:\, E:\, A:\, e por aí vai). Outros volumes no Linux são “pendurados” nesta raiz única iniciada em “/”, e isso traz uma série de vantagens. O termo exato, na verdade, é que os “volumes são montados”, mas isso é um assunto para outro capítulo.
/bin	É o diretório dos binários no Linux. Os comandos que podem ser executados de alguma forma por qualquer usuário ficam por aqui.
/boot	Armazena os arquivos necessários para realizar o boot do sistema Linux.
/dev	“dev” é uma abreviação da palavra inglesa devices, ou dispositivos; isso é bem diferente do Windows, pois por lá temos uma lista de dispositivos no Painel de Controle; em sistemas *nix, os dispositivos se tornam arquivos, fazendo parte de um sistema de arquivos normalmente. Sendo assim, periféricos como mouses, impressoras, webcams e dispositivos embarcados como HDs e suas partições viram um arquivo cada um em /dev.
/etc	Os arquivos de configuração ficam aqui, especialmente os de serviços da máquina; quando a configuração é específica para um determinado usuário, geralmente ficam em pastas ocultas (iniciadas com “.”, ponto) no diretório do usuário.
/home	Armazena os diretórios de usuários, sendo equivalente ao C:\Usuários ou C:\Users. Cada usuário possui seu próprio diretório (Por exemplo, meu usuário “hpoyatos” possui o /home/hpoyatos).
/lib e /lib64	Equivalente ao C:\Windows\System e C:\Windows\System32, são os diretórios onde ficam as bibliotecas de sistema; as bibliotecas em 64 bits ficam em /lib64, enquanto as de 32 bits ficam em /lib.
/lost+found	Diretório de “achados e perdidos” do sistema. Em uma eventual falha no sistema dos arquivos, o comando fsck deve ser utilizado e, caso ache arquivos ou fragmentos dele que ele não sabe onde guardar, eles ficam armazenados por aqui.
/media e /mnt	Diretórios de “montagem” de volumes, como pen-drives, CD-ROMs e DVD-ROMs.
/opt	Diretório opcional, pode ser utilizado como diretório alternativo de instalação de programas.
/proc	Diretório virtual de informações do sistema; por intermédio de seus pseudoarquivos, podemos obter informações sobre o processador, uso de memória, entre outros. No Windows é bem diferente, estas informações ficam, mais uma vez, no Painel de Controle.
/root	Diretório pessoal do superusuário ou root.
/run	Os processos em andamento (rodando ou <i>run</i>) geram arquivos .pid que ficam armazenados por aqui. Dúvidas sobre processos? Retorne ao capítulo 8 desta fase, “S.O., a alma digital”.
/sbin	Os binários (comandos) específicos para uso do superusuário (root) ficam armazenados aqui.
/srv	Dados de serviços em execução ficam armazenados neste diretório.
/sys	(apenas para kernels 2.13.x ou superiores) armazena os módulos para equipamentos USB.
/tmp	É um diretório de arquivos temporários que podem ser gerados por serviços ativos, equivalente ao C:\Windows\Temp.
/usr	É aqui que os programas são instalados. Assim sendo, seria equivalente ao C:\Arquivos de programas ou C:\Program Files.
/var	O diretório var armazena informações de tamanhos variados; é nele, por exemplo, que os arquivos que armazenam dados em um banco de dados ficam.
/var/log	Como também é uma informação de tamanho variado, os <i>logs</i> de sistema ficam neste subdiretório.

Quadro 1 – Diretórios em um sistema Linux
Fonte: Elaborado pelo autor (2018)

6.2 ls – Listar arquivos e diretórios

O comando **ls** permite exibir informações dos arquivos e subdiretórios de um determinado diretório:

```
hpoyatos@debian:~$ ls
Área de trabalho  Arquivo2.txt  Documentos  Imagens  Música  Vídeos
Arquivo1.txt      Arquivo3.txt  Downloads   Modelos  Público
```

Figura 16 – Exemplo de comando ls (simples)
Fonte: Elaborado pelo autor (2018)

Em sua execução simples, o comando exibirá apenas o nome dos arquivos e diretórios. Graças ao padrão de cores, podemos diferenciar o que é arquivo (em verde) do que é diretório (em azul). Não se baseie, no entanto, em extensões de arquivos: diferente do que acontece no Microsoft Windows, elas são completamente dispensáveis por aqui, embora sejam utilizadas como convenção. A extensão “txt” nos três arquivos do exemplo é apenas para fácil identificação e para seu maior conforto.

O comando **ls** é geralmente executado com dois parâmetros muito úteis: “-l”, que exibe uma lista longa ou detalhada de informações, e o “-a”, que vem de *all*, ou seja, traga **todos** os arquivos e diretórios, mesmo aqueles que são ocultos:

```
hpoyatos@debian:~$ ls -la
total 76
drwxr-xr-x 15 hpoyatos hpoyatos 4096 fev  3 22:17 .
drwxr-xr-x  3 root      root      4096 fev  3 20:47 ..
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Área de trabalho
-rw-r--r--  1 hpoyatos hpoyatos   0 fev  3 22:17 Arquivo1.txt
-rw-r--r--  1 hpoyatos hpoyatos   0 fev  3 22:17 Arquivo2.txt
-rw-r--r--  1 hpoyatos hpoyatos   0 fev  3 22:17 Arquivo3.txt
-rw-r--r--  1 hpoyatos hpoyatos  220 fev  3 20:47 .bash_logout
-rw-r--r--  1 hpoyatos hpoyatos 3526 fev  3 20:47 .bashrc
drwx-----  9 hpoyatos hpoyatos 4096 fev  3 21:38 .cache
drwx----- 10 hpoyatos hpoyatos 4096 fev  3 21:37 .config
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Documentos
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Downloads
drwx-----  3 hpoyatos hpoyatos 4096 fev  3 21:37 .gnupg
-rw-----  1 hpoyatos hpoyatos  314 fev  3 21:14 .ICEauthority
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Imagens
drwx-----  3 hpoyatos hpoyatos 4096 fev  3 21:14 .local
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Modelos
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Música
-rw-r--r--  1 hpoyatos hpoyatos  675 fev  3 20:47 .profile
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Público
drwx-----  2 hpoyatos hpoyatos 4096 fev  3 21:37 .ssh
drwxr-xr-x  2 hpoyatos hpoyatos 4096 fev  3 21:14 Vídeos
```

Figura 17 – Exemplo de comando ls com parâmetros “l” e “a”
Fonte: Elaborado pelo autor (2018)

Repare que vários outros arquivos e diretórios fizeram parte da listagem agora; seguindo o padrão do Unix, arquivos e diretórios ocultos são precedidos por “.” (ponto). Uma grande variedade de informações é trazida nesta lista, sendo:

- **Primeira coluna – Tipo de arquivo e permissionamento:** contendo sempre 10 caracteres, os primeiros deles se referem ao tipo de arquivo: um arquivo comum é representado por um traço “-“, e um diretório pela letra “d”. Você verá ainda as letras “l” (arquivo de link), “b” e “c” (para dispositivos, que transmitem por bloco ou “b” e caracter “c”). Os nove caracteres restantes devem ser lidos em conjuntos de três em três: os três primeiros são as permissões do dono do arquivo; os três do meio permissões do grupo dono do arquivo; e os três finais, dos demais. As letras “rwx” simbolizam leitura (r de read), escrita (w de write) e execução (x de execute).
- **Segunda coluna – Quantidade de arquivos:** indica a quantidade de arquivos ou diretórios aquele elemento possui. Repare que arquivos possuem sempre o número “1”, enquanto diretórios, geralmente, têm valores maiores. Na imagem “Exemplo de comando ls com parâmetros “l” e “a””, “.config” possui 10 arquivos ou subdiretórios dentro dele.
- **Terceira coluna:** exibe o nome do dono do arquivo. Quando o nome não estiver disponível, aponta o número identificador do usuário (uid).
- **Quarta coluna:** mostra o nome do grupo dono do arquivo; quando o nome não estiver disponível, exibe o número identificador do grupo (gid).
- **Quinta coluna:** indica o tamanho em bytes do arquivo. Ao acrescentar o parâmetro “h” (ls -lah) no comando, o tamanho fica mais legível para humanos, sendo apresentado em megabytes, gigabytes ou a unidade de medida mais próxima.
- **Sexta coluna:** mostra a data da última modificação no arquivo ou diretório.
- **Sétima coluna:** o nome do arquivo ou diretório propriamente dito.

Quando o diretório do qual queremos a lista não é mencionado, o comando automaticamente exibe o conteúdo do diretório atual (em meu caso, o diretório de meu usuário, /home/hpoyatos). O diretório desejado pode ser indicado logo depois dos parâmetros (ou antes, por que não?):

```
hpoyatos@debian:~$ ls -lah /
total 96K
drwxr-xr-x 23 root root 4,0K fev  3 20:20 .
drwxr-xr-x 23 root root 4,0K fev  3 20:20 ..
drwxr-xr-x  2 root root 4,0K fev  3 20:19 bin
drwxr-xr-x  3 root root 4,0K fev  3 20:21 boot
drwx----- 2 root root 4,0K fev  3 20:20 .cache
drwxr-xr-x 17 root root 2,9K fev  3 20:56 dev
drwxr-xr-x 118 root root 12K fev  3 22:01 etc
drwxr-xr-x  3 root root 4,0K fev  3 20:47 home
lrwxrwxrwx  1 root root   29 fev  3 19:46 initrd.img -> boot/initrd.img-4.9.0-4-amd64
4
lrwxrwxrwx  1 root root   29 fev  3 19:46 initrd.img.old -> boot/initrd.img-4.9.0-4-amd64
amd64
drwxr-xr-x 15 root root 4,0K fev  3 20:20 lib
drwxr-xr-x  2 root root 4,0K fev  3 19:45 lib64
drwx----- 2 root root 16K fev  3 19:45 lost+found
drwxr-xr-x  3 root root 4,0K fev  3 19:45 media
drwxr-xr-x  2 root root 4,0K fev  3 19:45 mnt
drwxr-xr-x  2 root root 4,0K fev  3 19:45 opt
dr-xr-xr-x 140 root root   0 fev  3 22:10 proc
drwx----- 3 root root 4,0K fev  3 21:42 root
drwxr-xr-x 20 root root 620 fev  3 22:01 run
drwxr-xr-x  2 root root 4,0K fev  3 20:55 sbin
drwxr-xr-x  2 root root 4,0K fev  3 19:45 srv
```

Figura 18 – Exemplo de comando ls com parâmetros “l”, “a” e “h” e diretório /
Fonte: Elaborado pelo autor (2018)

Aproveitando o “Exemplo de comando ls com parâmetros “l”, “a” e “h” e diretório /”, os arquivos do tipo “l”, initrd.img e initrd.img.old são meros atalhos para seus arquivos originais, armazenados do subdiretório /boot, como fica claro na listagem.

6.3 pwd – Mostrar diretório ou pasta atual

Exibe o diretório atual (pwd – *print work directory*):

```
hpoyatos@debian:~$ pwd
/home/hpoyatos
hpoyatos@debian:~$
```

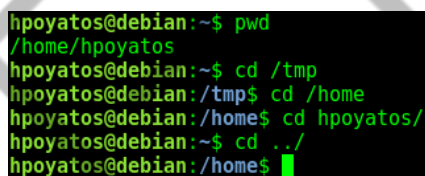
Figura 19 – Exemplo de comando pwd
Fonte: Elaborado pelo autor (2018)

É um comando pertinente especialmente em situações em que o *prompt* de comandos não exibe o diretório atual. No exemplo em questão, possui um *prompt* que mostra o diretório, mas de maneira resumida: “~” será sempre o diretório-padrão ou home do usuário que está sendo utilizado.

6.4 cd – Mudar de diretório ou pasta

O comando é um acrônimo de *change directory* e permite mudar o diretório atual. O diretório pode ser mencionado tanto com seu caminho completo ou absoluto (/home/hpoyatos) quanto o caminho relativo, que recebe este nome porque é referente ao diretório que estamos naquele momento. Ou seja, `cd hpoyatos` (sem a barra antes do hpoyatos), estando no diretório /home, possibilitará acessar o subdiretório hpoyatos, /home/hpoyatos, enquanto `cd /hpoyatos` tentaria entrar em um subdiretório hpoyatos na raiz do sistema de arquivos, o que não daria nada certo, afinal, este diretório não existe.

O comando `cd ..` indica o acesso ao diretório superior ou pai (simbolizado pelo ponto-ponto). A chamada do comando `cd` sem nenhuma informação adicional resulta na troca para o diretório-padrão do usuário, ou seja, equivale ao comando `cd ~` ou, no meu caso, `cd /home/hpoyatos`.

A terminal window with a black background and green text. The prompt is 'hpoyatos@debian:~\$'. The user enters 'pwd' and the output is '/home/hpoyatos'. Then the user enters 'cd /tmp' and the prompt changes to 'hpoyatos@debian:/tmp\$'. Next, the user enters 'cd /home' and the prompt changes to 'hpoyatos@debian:/home\$'. Then the user enters 'cd hpoyatos/' and the prompt changes to 'hpoyatos@debian:~/hpoyatos\$'. Finally, the user enters 'cd ../' and the prompt changes back to 'hpoyatos@debian:~/hpoyatos\$'.

```
hpoyatos@debian:~$ pwd
/home/hpoyatos
hpoyatos@debian:~$ cd /tmp
hpoyatos@debian:/tmp$ cd /home
hpoyatos@debian:/home$ cd hpoyatos/
hpoyatos@debian:~/hpoyatos$ cd ../
hpoyatos@debian:~/hpoyatos$
```

Figura 20 – Exemplo de comando cd
Fonte: Elaborado pelo autor (2018)

6.5 mkdir – Criar um diretório

O comando **mkdir** é uma abreviação da frase inglesa “*make directory*”, que significa, literalmente, “criar diretório”. Veja um exemplo de seu funcionamento:

```
hpoyatos@debian:~$ mkdir Nova\ Pasta
hpoyatos@debian:~$
```

Figura 21 – Exemplo de comando mkdir
Fonte: Elaborado pelo autor (2018)

Repare que caracteres especiais (como o espaço “ ” ou a própria barra invertida) são permitidos, desde que sejam precedidos por uma barra invertida. Relembro que o Linux é *case-sensitive*. Sendo assim, a pasta criada no exemplo foi “Nova Pasta” e não “nova pasta” (e muito menos NoVa PaStA).

Outra consideração é o fato de que o comando não me congratulou por ter escrito o comando corretamente; não acontece uma mensagem “Diretório criado com sucesso”. O sistema operacional só apresenta uma mensagem em caso de fracasso, ou seja, não espere um “parabéns” por ter acertado o comando (o Linux provavelmente acredita que “não fiz mais do que minha obrigação”).

Um parâmetro muito útil para acompanhar o mkdir é o “-p”, especialmente em situações que eu esteja criando subdiretórios, veja um exemplo:

```
$ mkdir -p Nova\ Pasta2/subpasta/subsubpasta
```

Comando de prompt 7 – Exemplo de uso do parâmetro -p no comando mkdir
Fonte: Elaborado pelo autor (2018)

Presumindo que eu esteja em meu diretório de usuário (/home/hpoyatos), o comando em questão criou uma pasta “Nova Pasta2”, um subdiretório “subpasta” dentro desta e um subdiretório “subsubpasta” dentro de “subpasta”. Sem o parâmetro “-p”, o comando apresentaria um erro, pois ele tentaria localizar a “Nova Pasta2” e não encontraria, ou seja, `mkdir Nova\Pasta2/subpasta/subsubpasta` (sem o “-p”) só criaria o diretório “subsubpasta”, tendo como condição /home/hpoyatos/Nova Pasta2/subpasta como um caminho existente e válido.

6.6 rm – Apagar um arquivo ou diretório

O comando `rm` é uma abreviação para o verbo inglês *to remove* e permitirá que apaguemos um arquivo ou um diretório. Embora exista o comando `rmdir`, restrito apenas para a remoção de diretórios, vamos nos ater ao comando que apaga qualquer um dos dois.

```
$ rm /home/hpoyatos/Arquivo1.txt
```

Comando de prompt 8 – Exemplo do comando `rm` apagando um arquivo

Fonte: Elaborado pelo autor (2018)

O comando apresentado remove o arquivo de texto “Arquivo1.txt” do diretório de usuário /home/hpoyatos. Repare que utilizei o caminho absoluto que, embora seja mais longo, possui menos risco de fazer algo errado. Posso rodar o comando `rm Arquivo1.txt` também, desde que eu tenha como diretório atual /home/hpoyatos; no entanto, se eu estiver em outro diretório que também possui um Arquivo13.txt, a confusão está criada.

Posso apagar diretórios, desde que o diretório em questão esteja completamente vazio (a regra vale para `rmdir` também). Ou seja:

```
$ rm /home/hpoyatos/Nova\Pasta2/subpasta/subsubpasta
```

Comando de prompt 9 – Exemplo do comando `rm` apagando um diretório vazio

Fonte: Elaborado pelo autor (2018)

Na Comando de prompt **\$ rm /home/hpoyatos/Nova Pasta2/subpasta/subsubpasta**, apenas o diretório “subsubpasta” seria apagado da existência, mantendo “Nova Pasta2” e “Nova Pasta2/subpasta” intactos. Apagar “Nova Pasta2” com o comando `rm ~/Nova\ Pasta2` (o til abrevia /home/hpoyatos !) não daria certo, pois o diretório não está vazio neste momento, ele tem “subpasta” dentro dele.

Para facilitar, informo que o comando `rm` possui dois parâmetros muito usados: “-r” significa “recursivo”, ou seja, se ao apagar “Nova Pasta2” ele tiver uma resistência, ele entra nos subdiretórios e os apaga primeiro, e também seus subsubdiretórios, caso existam, provocando um verdadeiro efeito cascata.

Como se trata de um comando perigoso, o Linux fará uma pergunta para configurar cada diretório que eu tentar apagar, esperando um “y” indicando “yes” ou “sim”. Como apagar um diretório com outros 30 dentro dele se torna uma tarefa muito chata, a maioria adiciona um “-f” de *force*, ou seja, “força uma barra” e não me peça confirmação, e aí o comando fica realmente perigoso:

```
$ rm -rf /home/hpoyatos/Nova\ Pasta2
```

Comando de prompt 10 – Exemplo do comando `rm` apagando um diretório cheio

Fonte: Elaborado pelo autor (2018)

A Comando de prompt **\$ rm -rf /home/hpoyatos/Nova\ Pasta2** apaga o diretório “Nova Pasta2” e absolutamente tudo que estiver dentro dele, sem dó nem piedade; havia apenas um subdiretório “subpasta” vazio dentro dele, mas poderiam ter outros 5 milhões de arquivos que não faria a menor diferença (na verdade, faria, mas o comando demoraria um pouquinho).

“Se eu me arrepender, eu restauro da Lixeira”, não, não restaura! Lixeira é para usuários de interface gráfica que não sabem o que estão fazendo. Você sabe o que está fazendo, ou pelo menos deveria.

O comando só permite que eu apague arquivos e diretórios dos quais eu tenha permissão de fazê-lo, ou seja, o dano pode ser contido, a não ser que você seja o superusuário root.

Ou seja, o comando `# rm -rf /`, que não vou colocar em um quadro porque não quero que você execute, apagaria, com privilégios totais, o sistema de arquivos inteiro. Ok, na verdade, ele apagaria todos os arquivos e diretórios que encontrasse pela frente, até apagar o arquivo `/bin/rm`, para, no momento seguinte, não saber mais o que fazer (acredite em mim, eu já rodei, embora não me orgulhe disso). No entanto, o estrago generalizado está feito. Sendo assim, “grandes poderes trazem grandes responsabilidades”.

6.7 cp – Copiando arquivos e diretórios

O comando **cp** é uma abreviatura do verbo inglês *to copy* e permite copiar arquivos e diretórios, solicitando sempre qual o arquivo copiado e, logo depois, para onde ele será copiado:

```
$ cp /home/hpoyatos/Arquivo2.txt /tmp
```

Comando de prompt 11 – Exemplo do comando cp copiando um arquivo

Fonte: Elaborado pelo autor (2018)

No exemplo em Comando de prompt **\$ cp /home/hpoyatos/Arquivo2.txt /tmp**, o arquivo Arquivo2.txt foi copiado para outro diretório, o diretório de arquivos temporários `/tmp`. É importante que você tenha permissão de escrita no diretório destino e, neste caso, `/tmp` permite a qualquer usuário de sistema escrever arquivos.

```
$ cp /home/hpoyatos/Arquivo2.txt /tmp/NovoNome2.txt
```

Comando de prompt 12 – Exemplo do comando cp copiando um arquivo, renomeando-o

Fonte: Elaborado pelo autor (2018)

O segundo exemplo em Comando de prompt **\$ cp /home/hpoyatos/Arquivo2.txt /tmp/NovoNome2.txt** faz a mesma operação, mas ao copiar renomeia o novo arquivo para “NovoNome2.txt”. A primeira execução preservou o nome original, ou seja, `/tmp/Arquivo2.txt` existe.

Copiar diretórios inteiros é perfeitamente possível, mas precisamos de ajuda de três parâmetros: “-R” indica recursividade, ou seja, permite copiar o diretório e tudo contido dentro dele; o parâmetro “-v” vem da palavra em inglês *verbose* e é muito comum em comandos Linux, neste caso, mostrará quais arquivos dentro do diretório foram copiados; conheça também “-p”, que preserva as permissões originais do arquivo copiado:

```
hpoyatos@debian:~$ cp -Rvp /home/hpoyatos/Imagens /tmp
'/home/hpoyatos/Imagens' -> '/tmp/Imagens'
'/home/hpoyatos/Imagens/Imagem1.png' -> '/tmp/Imagens/Imagem1.png'
hpoyatos@debian:~$
```

Figura 22 – Exemplo de comando cp copiando diretório
Fonte: Elaborado pelo autor (2018)

O comando apresentado na Figura “Exemplo de comando cp copiando diretório” copiou o diretório Imagens com todo seu conteúdo (graças ao “-v”, vemos que apenas um arquivo constava, “Imagem1.png”) para o diretório /tmp.

6.8 mv – Movendo ou renomeando arquivos e diretórios

O comando **mv** é uma abreviatura do verbo inglês *to move* e permite mover arquivos e diretórios de um ponto a outro, solicitando sempre a origem e depois o destino:

```
$ mv /tmp/Imagens /opt
```

Comando de prompt 13 – Exemplo do comando mv usado para mover um diretório
Fonte: Elaborado pelo autor (2018)

No exemplo na Comando de prompt **\$ mv /tmp/Imagens /opt**, o diretório Imagens/ presente no diretório de arquivos temporários /tmp foi transferido para o diretório /opt; para dar certo, o usuário que realiza a operação precisa ter permissão de escrita no diretório destino. O processo para arquivos é idêntico.

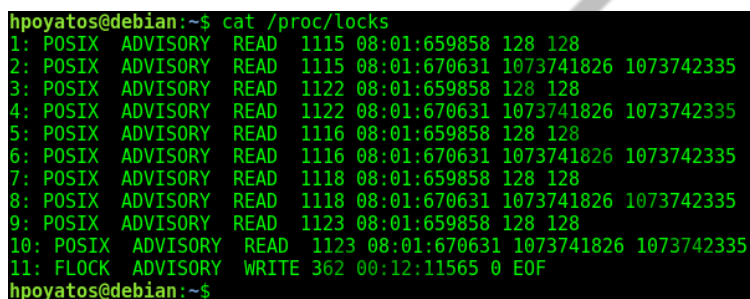
Para renomear o arquivo, o procedimento é similar; basta informar o nome atual do arquivo e, na sequência, o novo nome (faça o mesmo para diretórios):

```
$ mv /tmp/Arquivo2.txt /tmp/Arquivo1.txt
```

Comando de prompt 14 – Exemplo do comando mv usado para renomear um arquivo
Fonte: Elaborado pelo autor (2018)

6.9 cat – Exibe o conteúdo de um arquivo

O comando **cat** pode ser utilizado para exibir o conteúdo de um arquivo texto no terminal de comandos e deve ser acompanhado pelo caminho do arquivo.



```
hpoyatos@debian:~$ cat /proc/locks
1: POSIX ADVISORY READ 1115 08:01:659858 128 128
2: POSIX ADVISORY READ 1115 08:01:670631 1073741826 1073742335
3: POSIX ADVISORY READ 1122 08:01:659858 128 128
4: POSIX ADVISORY READ 1122 08:01:670631 1073741826 1073742335
5: POSIX ADVISORY READ 1116 08:01:659858 128 128
6: POSIX ADVISORY READ 1116 08:01:670631 1073741826 1073742335
7: POSIX ADVISORY READ 1118 08:01:659858 128 128
8: POSIX ADVISORY READ 1118 08:01:670631 1073741826 1073742335
9: POSIX ADVISORY READ 1123 08:01:659858 128 128
10: POSIX ADVISORY READ 1123 08:01:670631 1073741826 1073742335
11: FLOCK ADVISORY WRITE 362 00:12:11565 0 EOF
hpoyatos@debian:~$
```

Figura 23 – Exemplo de comando cat exibindo o conteúdo do arquivo /proc/locks
Fonte: Elaborado pelo autor (2018)

6.10 head – Exibir linhas iniciais de um arquivo

Como o nome indica, o comando **head** permite exibir o cabeçalho do arquivo, ou seja, é o comando **cat** que mostra apenas as linhas iniciais do arquivo; pode ser muito útil para arquivos muito grandes dos quais você só precisa visualizar as primeiras linhas.

A Figura 23 utiliza o parâmetro “-n”, que permite determinar o número de linhas iniciais que desejo ver do arquivo e está sendo usado para exibir as dez primeiras linhas do arquivo **/proc/cpuinfo** (que informa dados do processador):


```
hpoyatos@debian:~$ head -n 10 /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 61
model name     : Intel(R) Core(TM) i7-5500U CPU @ 2.40GHz
stepping       : 4
cpu MHz        : 2394.458
cache size     : 4096 KB
physical id    : 0
siblings       : 1
hpoyatos@debian:~$
```

Figura 24 – Exemplo de comando head exibindo as dez primeiras linhas do arquivo /proc/cpuinfo
Fonte: Elaborado pelo autor (2018)

6.11 tail – Exibir linhas finais de um arquivo

O comando **tail** faz o inverso do comando **head**, e a palavra significa em inglês “cauda”: exibe as últimas linhas do arquivo desejado. O parâmetro “-n” também pode ser usado aqui para definir o número de linhas finais que deseja exibir.

Muito utilizado para a leitura de arquivos de log, que geralmente são muito grandes e, em vários casos, desejamos apenas os últimos acontecimentos do sistema. O exemplo da Figura “Exemplo de comando tail exibindo as dez últimas linhas do arquivo /var/log/messages” mostra as dez últimas linhas do principal arquivo de log de sistema, /var/log/messages (com sudo, pois são necessários privilégios especiais):

```
hpoyatos@debian:~$ sudo tail -n 10 /var/log/messages
[sudo] senha para hpoyatos:
Feb  7 22:55:31 debian pulseaudio[1018]: [pulseaudio] alsa-util.c: Disabling timer-based scheduling because running inside a VM.
Feb  7 22:55:32 debian gnome-shell[992]: JS LOG: Failed to launch ibus-daemon: Falha ao executar processo filho "ibus-daemon" (Arquivo ou diretório não encontrado)
Feb  7 22:55:32 debian goa-daemon[1037]: goa-daemon version 3.22.5 starting
Feb  7 22:55:33 debian gnome-session-binary[914]: Entering running state
Feb  7 22:55:33 debian tracker-store.desktop[1117]: (uint32 1,)
Feb  7 22:55:33 debian gnome-settings-[1091]: g_task_return_error: assertion 'error != NULL' failed
Feb  7 22:55:34 debian gnome-settings-[1091]: failed to get edid: unable to get EDID for output
Feb  7 22:55:34 debian gnome-settings-[1091]: unable to get EDID for xrandr-default: unable to get EDID for output
Feb  7 22:55:34 debian gnome-settings-[602]: unable to get EDID for xrandr-default: unable to get EDID for output
Feb  7 22:55:35 debian gnome-shell[992]: GNOME Shell started at Wed Feb 07 2018 22:55:32 GMT-0200
hpoyatos@debian:~$
```

Figura 25 – Exemplo de comando tail exibindo as dez últimas linhas do arquivo /var/log/messages
Fonte: Elaborado pelo autor (2018)

Aproveitando, é muito comum utilizar o comando `tail` com um parâmetro “-f” (de *follow*) que, como o nome indica, a execução acompanha o crescimento do arquivo. Em outras palavras, o comando `tail` “prende” o terminal, esperando novas linhas do arquivo surgirem e serem exibidas. É o comando ideal para acompanhar arquivos de log de sistema. Quando deseja voltar o terminal, basta utilizar o atalho [CTRL]+[C] no teclado.

```
hpoyatos@debian:~$ tail -n 5 -f /var/log/syslog
Oct 31 13:58:56 ubuntu-vmware gnome-shell[1703]: polkitAuthenticationAgent: Received 2 identities that can
be used for authentication. Only considering one.
Oct 31 13:58:58 ubuntu-vmware org.gnome.Shell.desktop[1703]: polkit-agent-helper-1: pam_authenticate failed
: Authentication failure
Oct 31 13:59:01 ubuntu-vmware AptDaemon.Worker: INFO: Processing transaction /org/debian/apt/transaction/ad
dd98d844ee4ffdb25ee48a63ff3dde
Oct 31 13:59:01 ubuntu-vmware org.debian.apt[747]: 13:59:01 AptDaemon.Worker [INFO]: Processing transaction
/org/debian/apt/transaction/add98d844ee4ffdb25ee48a63ff3dde
Oct 31 13:59:01 ubuntu-vmware systemd-resolved[26597]: Server returned error NXDOMAIN, mitigating potential
DNS violation DVE-2018-0001, retrying transaction with reduced feature level UDP.
```

Figura 26 – Exemplo de comando `tail` acompanhando `/var/log/syslog` com o parâmetro -f
Fonte: Elaborado pelo autor (2018)

6.12 less – Exibe o conteúdo de um arquivo

O comando **less** pode ser muito útil para visualizar arquivos muito grandes, pois, com ele, podemos navegar no conteúdo do arquivo com as setas para cima e para baixo, pois a exibição com **cat** é de difícil navegação (para **cat**, use [CTRL]+[PAGE UP] e [CTRL]+[PAGE DOWN]). Como o comando **less** trava o terminal, digite “q” (de quit) para sair do comando, a exemplo do que acontece com o comando **man**. A Figura “Exemplo de comando **less** exibindo o arquivo `/var/log/messages`” é o resultado do comando `$ sudo less /var/log/messages`:

```
Oct 31 12:22:01 ubuntu-vmware systemd-resolved[694]: Server returned error NXDOMAIN, mitigating potential D
NS violation DVE-2018-0001, retrying transaction with reduced feature level UDP.
Oct 31 12:22:05 ubuntu-vmware kernel: [80074.914063] Buffer I/O error on dev sdb1, logical block 121667584,
lost sync page write
Oct 31 12:22:05 ubuntu-vmware kernel: [80074.914067] JBD2: Error -5 detected when updating journal superblo
ck for sdb1-8.
Oct 31 12:22:05 ubuntu-vmware kernel: [80074.914068] Aborting journal on device sdb1-8.
Oct 31 12:22:05 ubuntu-vmware kernel: [80074.914069] Buffer I/O error on dev sdb1, logical block 121667584,
lost sync page write
Oct 31 12:22:05 ubuntu-vmware kernel: [80074.914070] JBD2: Error -5 detected when updating journal superblo
ck for sdb1-8.
:
```

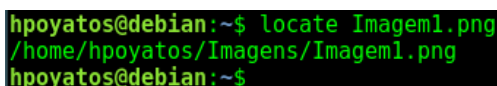
Figura 27 – Exemplo de comando **less** exibindo o arquivo `/var/log/messages`
Fonte: Elaborado pelo autor (2018)

6.13 locate – Localiza arquivos

Por vezes, é difícil localizar um arquivo armazenado em um sistema de arquivos. Com o objetivo de facilitar esta tarefa, podemos usar o comando **locate**.

Primeiramente, é necessário instalar o pacote do comando digitando o comando `$ sudo apt install locate`, pois ele não vem por padrão na distribuição Debian. Uma vez instalado, é necessário criar a base para buscas e, para tal, é necessário rodar o comando `$ sudo updatedb` de tempos em tempos (é comum agendar a execução automática deste comando usando o cron).

Para utilizar o comando, digite **locate** e o nome do arquivo que você deseja buscar:



```
hpoyatos@debian:~$ locate Imagem1.png
/home/hpoyatos/Imagens/Imagem1.png
hpoyatos@debian:~$
```

Figura 28 – Exemplo de comando locate usado para localizar o arquivo Imagem1.png
Fonte: Elaborado pelo autor (2018)

6.14 grep – Localiza termos dentro de arquivos

Trata-se de um excelente comando para realizar buscas de palavras-chave dentro de arquivos; basta digitar a palavra-chave logo após o comando, seguido do local que se deseja procurar. Na Figura “Exemplo de comando grep buscando a palavra fiap dentro do arquivo /var/log/auth.log”, o comando foi utilizado para buscar a palavra “fiap” dentro do arquivo `/var/log/auth.log`, responsável por gerenciamento de usuários:

```
hpoyatos@debian:~$ sudo grep fiap /var/log/auth.log
Feb  8 00:12:53 debian sudo: hpoyatos : TTY=pts/0 ; PWD=/home/hpoyatos ; USER=ro
ot ; COMMAND=/usr/sbin/useradd fiap
Feb  8 00:12:53 debian useradd[1941]: new group: name=fiap, GID=1002
Feb  8 00:12:53 debian useradd[1941]: new user: name=fiap, UID=1002, GID=1002, h
ome=/home/fiap, shell=
Feb  8 00:15:17 debian sudo: hpoyatos : TTY=pts/0 ; PWD=/home/hpoyatos ; USER=ro
ot ; COMMAND=/usr/sbin/useradd fiap -d /home/fiap -p senha usuario fiap
Feb  8 00:15:17 debian useradd[2072]: failed adding user 'fiap', data deleted
Feb  8 00:15:34 debian sudo: hpoyatos : TTY=pts/0 ; PWD=/home/hpoyatos ; USER=ro
ot ; COMMAND=/usr/sbin/useradd fiap -d /home/fiap -p senha usuario fiap
Feb  8 00:15:34 debian useradd[2096]: new group: name=fiap, GID=1002
Feb  8 00:15:34 debian useradd[2096]: new user: name=fiap, UID=1002, GID=1002, h
ome=/home/fiap, shell=
Feb  8 00:22:09 debian sudo: hpoyatos : TTY=pts/0 ; PWD=/home/hpoyatos ; USER=ro
ot ; COMMAND=/usr/sbin/groupadd fiapon
Feb  8 00:22:09 debian groupadd[2114]: group added to /etc/group: name=fiapon, G
ID=1003
Feb  8 00:22:09 debian groupadd[2114]: group added to /etc/gshadow: name=fiapon
Feb  8 00:22:09 debian groupadd[2114]: new group: name=fiapon, GID=1003
Feb  8 09:56:17 debian sudo: hpoyatos : TTY=pts/0 ; PWD=/home/hpoyatos ; USER=ro
ot ; COMMAND=/usr/sbin/usermod -aG fiapon,sudo -e 2018-31-12 fiap
Feb  8 09:56:17 debian usermod[2288]: change user 'fiap' expiration from 'never'
to '2020-07-12'
Feb  8 09:56:17 debian usermod[2288]: add 'fiap' to group 'sudo'
Feb  8 09:56:17 debian usermod[2288]: add 'fiap' to group 'fiapon'
Feb  8 09:56:17 debian usermod[2288]: add 'fiap' to shadow group 'sudo'
Feb  8 09:56:17 debian usermod[2288]: add 'fiap' to shadow group 'fiapon'
```

Figura 29 – Exemplo de comando grep buscando a palavra fiap dentro do arquivo /var/log/auth.log
Fonte: Elaborado pelo autor (2018)

O comando **grep** permite procurar termos em diretórios completos. O exemplo na Figura “Exemplo de comando grep buscando o termo FIAP ON em /home” busca pelo termo “FIAP ON” dentro de todos os diretórios contidos em /home, graças ao parâmetro “-r”, que indica recursividade (sudo é necessário para vasculhar no diretório de outros usuários:

```
hpoyatos@debian:~$ sudo grep FIAP\ ON /home -r
/home/hpoyatos/Arquivo2.txt:0 FIAP ON é muito legal!
/home/hpoyatos/Arquivo3.txt:0s comandos do sistema operacional Linux são legais!
Aprendi no FIAP ON!
hpoyatos@debian:~$
```

Figura 30 – Exemplo de comando grep buscando o termo FIAP ON em /home
Fonte: Elaborado pelo autor (2018)

Repare que o exemplo achou ocorrências em dois arquivos diferentes, e a linha indica em qual arquivo o termo foi localizado.

6.15 unzip – Descompactar arquivos zip

Existirão ocasiões em que você precisará descompactar arquivos do tipo “zip”, ação vulgarmente conhecida como “deszipar”, e o comando é este:

```
$ unzip -v arquivoZipado.zip
```

Comando de prompt 15 – Exemplo do comando unzip descompactando arquivos zip

Fonte: Elaborado pelo autor (2018)

O parâmetro “-v” permitirá listar cada arquivo e diretório que ele está descompactando. Sem o parâmetro, o comando faz o trabalho e devolve o *prompt* da maneira mais silenciosa possível. O parâmetro “-t” é útil e permite testar o arquivo antes de descompactá-lo. Assim sendo:

```
$ unzip -tv arquivoZipado.zip
```

Comando de prompt 16 – Exemplo do comando unzip testando um arquivo zip

Fonte: Elaborado pelo autor (2018)

A Comando de prompt **\$ unzip -tv arquivoZipado.zip** exibe todos os arquivos e diretórios de arquivoZipado.zip e testa a integridade dos arquivos sem, no entanto, realmente descompactá-los, o que pode ser muito útil em vários casos.

6.16 tar – (Des)empacotar e (des)comprimir arquivos

Comando absolutamente indispensável, tar permite empacotar e desempacotar arquivos, ou seja, vários arquivos se tornam um só e vice-versa. Repare que não usei os verbos “compactar” e “descompactar”: esta tarefa fica para outro comando, como é o caso do `gzip` (o mais utilizado) ou do `bzip2` (que compacta melhor). “Terei que empacotar com um comando e depois comprimir com outro?” Teria, mas nada que conhecer os parâmetros certos não resolva com um comando só.

Vamos a um exemplo na Figura “Exemplo de comando tar empacotando e comprimindo arquivos”:

```
hpoyatos@debian:~$ ls
Área de trabalho  Arquivo3.txt  Downloads  Modelos  Nova Pasta  Vídeos
Arquivo2.txt      Documentos    Imagens    Música    Público
hpoyatos@debian:~$ tar -czvf exemplo1.tar.gz *
Área de trabalho/
Arquivo2.txt
Arquivo3.txt
Documentos/
Downloads/
Imagens/
Imagens/Imagem1.png
Modelos/
Música/
Nova Pasta/
Público/
Vídeos/
hpoyatos@debian:~$
```

Figura 31 – Exemplo de comando tar empacotando e comprimindo arquivos
Fonte: Elaborado pelo autor (2018)

O exemplo começa com um “ls” para sabermos o que existia no diretório atual (no caso /home/hpoyatos). O comando tar pede primeiro o destino (o empacotamento) e depois os arquivos que farão parte deste pacote; (os arquivos que serão empacotados ou o local onde serão desempacotados, quando realizada a operação inversa).

O comando em questão (Figura “Exemplo de comando tar empacotando e comprimindo arquivos”) conta com quatro parâmetros: “-c” indica que o arquivo de pacote está sendo criado, enquanto “-z” aponta que desejo comprimir o arquivo de destino com gzip; “-v” é o sempre útil verbose, enquanto “-f”, desta vez, não é force e, sim, qual o nome do arquivo que irei gerar, neste caso, exemplo1.tar.gz.

Lembre-se que extensões são opcionais, mas não quer dizer que não devemos usá-las: o arquivo resultante poderia se chamar apenas exemplo1, mas seria complicado fazer o processo inverso sem saber a natureza do arquivo; a extensão .tar indica que o arquivo é um empacotamento do tipo tar, enquanto a segunda extensão do arquivo, .gz, aponta que este foi comprimido com gzip. É uma convenção útil, embora existam outros usuários que preferem a extensão .tgz.

Chamo a atenção para o fato de o comando ser recursivo: graças ao “-v”, podemos ver que todos os arquivos em /home/hpoyatos foram incluídos, e até mesmo arquivos presentes em subdiretório foram considerados, como /home/hpoyatos/Imagens/Imagem1.png.

```
$ tar -czvf exemplo2.tgz *.txt
```

Comando de prompt 17 – Exemplo do comando tar empacotando e comprimindo arquivos txt
Fonte: Elaborado pelo autor (2018)

Este segundo exemplo no Comando de prompt **\$ tar -czvf exemplo2.tgz *.txt** filtra os arquivos que serão incluídos no pacote; agora, apenas arquivos de extensão **.txt** serão usados, no caso, **Arquivo2.txt** e **Arquivo3.txt**.

O parâmetro “-j” no lugar do “-z” permitirá usar o compactador bzip2 no lugar do gzip e, neste caso, as extensões aceitáveis para arquivo final seriam **.tar.bz2** ou **.tbz**.

Para fazer o processo inverso:



```
hpooyatos@debian:~$ tar -xzf exemplo1.tar.gz
Área de trabalho/
Arquivo2.txt
Arquivo3.txt
Documentos/
Downloads/
Imagens/
Imagens/Imagem1.png
Modelos/
Música/
Nova Pasta/
Público/
Videos/
hpooyatos@debian:~$
```

Figura 32 – Exemplo de comando tar empacotando e comprimindo arquivos
Fonte: Elaborado pelo autor (2018)

Como visto na Figura “Exemplo de comando tar empacotando e comprimindo arquivos”, basta substituir o parâmetro “-c” por “-x” (do verbo inglês *to extract* ou extrair) e não informar os arquivos de origem (como ***** ou ***.txt**), pois, desta vez, a origem é o arquivo de pacote. A exemplo do que vimos em *unzip*, o parâmetro “-t” no lugar de “-x” permite testar a integridade do arquivo de pacote e ver seu conteúdo sem, no entanto, “explodir” (descompactar) o arquivo.

6.17 wget – Baixando arquivos da Internet

Em certas ocasiões, será necessário baixar arquivos de um algum *host* na Internet. Como fazer isso sem um navegador? Você pode usar o comando `wget`:

```
hpoyatos@debian:~$ wget https://cdn.kernel.org/pub/linux/kernel/v4.x/linux-4.15.
tar.xz
--2018-02-05 23:31:34-- https://cdn.kernel.org/pub/linux/kernel/v4.x/linux-4.15
.tar.xz
Resolvendo cdn.kernel.org (cdn.kernel.org)... 151.101.93.176, 2a04:4e42:16::432
Conectando-se a cdn.kernel.org (cdn.kernel.org)[151.101.93.176]:443... conectado
.
A requisição HTTP foi enviada, aguardando resposta... 200 OK
Tamanho: 102181404 (97M) [application/x-xz]
Salvando em: "linux-4.15.tar.xz"

linux-4.15.tar.xz  50%[=====>          ] 49,18M  1,51MB/s   eta 27s   ^
C
hpoyatos@debian:~$
```

Figura 33 – Exemplo de comando `wget` baixando arquivos da Internet
Fonte: Elaborado pelo autor (2018)

No exemplo em questão (“Exemplo de comando `wget` baixando arquivos da Internet”), decidi baixar o código-fonte da versão mais atual do Kernel do Linux (4.15, na ocasião) do site oficial, `kernel.org`. Uma curiosidade: a extensão `.xz` é utilizada para compactador `xz` (do pacote `xz-utils`), considerado mais eficaz que o `bzip2` (se instalado, basta utilizar o parâmetro “-J” no comando `tar` para fazer uso).

Quando metade do arquivo foi baixado, interrompi o processo de download com um `[CTRL]+[C]` (que cancela qualquer comando em andamento). Observe o que acontece quando repito o comando incluindo o parâmetro “-c” de *continue* (seta para cima e a tecla `[ENTER]` me poupam de digitá-lo novamente):


```
A requisição HTTP foi enviada, aguardando resposta... 200 OK
Tamanho: 102181404 (97M) [application/x-xz]
Salvando em: "linux-4.15.tar.xz"

linux-4.15.tar.xz  50%[=====>          ] 49,18M 1,51MB/s  eta 27s  ^
C
hpoyatos@debian:~$ wget -c https://cdn.kernel.org/pub/linux/kernel/v4.x/linux-4.
15.tar.xz
--2018-02-05 23:32:33--  https://cdn.kernel.org/pub/linux/kernel/v4.x/linux-4.15
.tar.xz
Resolvendo cdn.kernel.org (cdn.kernel.org)... 151.101.93.176, 2a04:4e42:16::432
Conectando-se a cdn.kernel.org (cdn.kernel.org)[151.101.93.176]:443... conectado
A requisição HTTP foi enviada, aguardando resposta... 206 Partial Content
Tamanho: 102181404 (97M), 50338012 (48M) restantes [application/x-xz]
Salvando em: "linux-4.15.tar.xz"

linux-4.15.tar.xz  68%[+++++++=>        ] 66,86M 1,74MB/s  eta 17s
```

Figura 34 – Exemplo de comando wget retomando um download pela metade
 Fonte: Elaborado pelo autor (2018)

O parâmetro permite retomar o download exatamente de onde parou, de sua metade. Muito útil, não é? Ah, e caso tenha se perguntado se precisa de uma interface gráfica com um navegador web para localizar o endereço desejado, não precisa: instale o navegador *web* para terminal de comando **links**, com o comando `sudo apt-get install links`.

CONCLUSÃO

Esperamos que este capítulo tenha atingido seu objetivo: o de apresentar os comandos essenciais deste fascinante sistema operacional. Embora este capítulo acabe por aqui, retornaremos a falar do sistema do pinguim em diversos capítulos ao longo deste curso. Não perca!

REFERÊNCIAS

FERREIRA, R. E. **Linux** – Guia do Administrador do Sistema. 2ª ed. São Paulo: Novatec, 2008.

GEORGE, A. **Margaret Hamilton Led the NASA Software Team That Landed Astronauts on the Moon.** 2019. Disponível em: <
<https://www.smithsonianmag.com/smithsonian-institution/margaret-hamilton-led-nasa-software-team-landed-astronauts-moon-180971575/>>. Acesso em: 11 dez. 2020.

GLOSSÁRIO

*nix	Termo utilizado para referenciar sistemas operacionais baseados em Unix, como o caso do Linux.
Kernel	É o coração de qualquer sistema operacional responsável por realizar o gerenciamento de processos, os recursos físicos do computador e implementar o sistema de arquivos. A interação do usuário com o Kernel é sempre indireta, via interface gráfica ou terminal de comandos.
SSH	<i>Secure Shell</i> ou <i>shell</i> seguro é um serviço de acesso remoto que permite realizar comandos em outra máquina (como o antigo <i>telnet</i>), mas que trafega as interações em um túnel criptografado.